# Too Many Hands on the Throttle: Arbitrating Overlapping Reliability Controls in Service Fleets

Abhishek Pandey
Meta
pandeyabhi1987@gmail.com

Abhinav Pandey
Cisco
pandey.aby@gmail.com

Abstract—A production service fleet runs an autoscaler, a load shedder, a circuit breaker, a concurrency limiter, a rate limiter, a timeout policy, a retry budget, and a router. These are separate control loops, owned by separate teams, configured through separate surfaces, and tuned in isolation. They all observe correlated signals and they all act on one plant. We show that this arrangement is not merely untidy but structurally unsound, and that the resulting failures are invisible to every controller involved.

We identify three coupling channels and characterize each. Actuation coupling arises when two actuators move the same plant variable: their gains superpose, and we show that controllers individually tuned inside the delay margin compose to an unstable loop, with the stability condition depending on a quantity no single controller's configuration surface exposes. Observation coupling arises when one controller's action corrupts another's input: we show that a fast load-reducing controller acting upstream of a slow provisioning controller's measurement point creates a positive feedback loop, and that detuning either controller reduces the loop gain without removing the loop. Topological coupling arises when an action redistributes load, and we give the subcriticality condition under which ejection does not cascade.

We then present Concerto, an arbitration layer with one principle per channel: a single severity estimate with a bounded authority budget allocated rather than summed across actuators, counterfactual reconstruction of each controller's input signal, and topology-gated admission of redistributive actions. We establish that the premise holds in deployed software through an audit of the configuration surfaces of Kubernetes, Envoy, Istio, and Linkerd, none of which exposes a coordination hook between these mechanisms.

We report the model, the architecture, the configuration audit method, and a full evaluation protocol over an open microservice testbed and a fleet simulator. The experimental campaign is in progress and no measurements are claimed here.

Index Terms—reliability control, autoscaling, load shedding, circuit breaking, control allocation, cascading failure, microservices

## I. Introduction

Ask an operator how their service protects itself under load and you will get a list. There is an autoscaler that adds replicas when utilization rises. There is a load shedder that rejects low-priority work when latency climbs. There is outlier detection that ejects unhealthy endpoints. There is a concurrency limiter that caps in-flight requests, a rate limiter that caps outbound calls, a timeout policy, a retry budget, and a router that moves traffic away from trouble.

Each of these is a feedback controller. Each was designed in isolation, documented in isolation, and tuned in isolation. Most are configured through different surfaces, and in a large organization most are owned by different teams. And every one of them observes some function of the same underlying congestion state and acts, directly or indirectly, on that same state.

Control engineers have a name for this situation. When several actuators have authority over one plant, the assignment of corrective effort among them is the control allocation problem, and it has been treated rigorously for decades in aerospace, where an aircraft has more control surfaces than axes of motion [1], [2]. Service fleets have the same structure and do not use the same discipline. We allocate nothing. We run every controller at full authority and hope.

This paper argues that the hope is misplaced in a specific and predictable way, and that the resulting failure modes are invisible from inside any individual controller.

### A. A failure no controller can see

Consider a fleet with an autoscaler and a load shedder, both configured with defaults that a careful operator would consider reasonable. Load rises. The shedder sees latency climb and begins rejecting requests, which is exactly its job, and latency recovers. The autoscaler, meanwhile, reads request rate at the service, sees it fall, concludes that demand has dropped, and scales the fleet down. The shedder now sees healthy latency on a smaller fleet and relaxes. Full offered load returns to reduced capacity. Latency spikes, the shedder clamps down harder, and the cycle repeats with a larger amplitude.

Every controller in this story behaved exactly as specified. The shedder shed under latency pressure. The autoscaler scaled to observed demand. Neither configuration is wrong. The fault is in the composition, and neither controller's telemetry contains enough information to attribute it, because each one sees only its own loop closing correctly.

### B. Contributions

1) A taxonomy of three coupling channels (Section III), distinguished by what is shared: the actuated

variable, the measured signal, or the load distribution. The distinction matters because each channel requires a different remedy and two of the three are untreatable by retuning.

2) A formal characterization of each channel (Section IV). Proposition 1 shows that controllers individually tuned inside the delay margin compose to instability once their margin fractions sum past unity, and Corollary 1 shows the governing quantity is not observable at any single controller. Proposition 2 shows that signal masking creates a positive feedback loop, and Corollary 2 shows detuning cannot remove it. Proposition 3 gives the subcriticality threshold for redistributive actions.
3) Evidence that the premise holds in deployed software (Section V). We audit the configuration surfaces of the mechanisms operators actually run and record, for each, the signal it reads, the plant variable it actuates, and whether any coordination hook with its peers exists. We pair this with the incident record and with the practitioner literature that already names these interactions.
4) Concerto, an arbitration layer (Section VI), with one design principle per coupling channel plus a timescale separation rule, and an argument that each principle addresses its channel structurally rather than by tuning.
5) An evaluation protocol (Section VIII) over an open microservice testbed and a fleet simulator, with per-principle ablations and an explicit statement of which baseline represents current practice.

We are explicit about scope. This paper delivers the taxonomy, the model, the audit method, the architecture, and the protocol. The experimental campaign is in progress. Section VIII states hypotheses and measurement plans and reports no results.

## II. Background

### A. The mechanisms

We use the following terms for the controllers a modern fleet runs. An autoscaler adjusts replica count from a utilization or demand signal. A load shedder or admission controller rejects a fraction of offered work, usually prioritized, when a congestion signal exceeds a threshold. A circuit breaker or outlier detector removes an endpoint from the serving set when its error rate or latency exceeds a threshold. A concurrency limiter caps in-flight requests. A rate limiter caps request rate. A timeout policy bounds per-call latency. A retry policy reissues failed calls, typically bounded by a retry budget. A router distributes traffic across endpoints and performs failover.

### B. Why the composition is not studied

Each mechanism has a substantial literature of its own. Autoscaling has been treated as a control problem [3]. Admission control and overload protection have a long lineage in both networking and web serving [4]. Adaptive concurrency limiting draws on congestion control [5]. Circuit breaking is folklore formalized into libraries. Each literature assumes its mechanism is the one closing the loop.

The composition is discussed mainly in practitioner writing. The Google SRE book devotes a chapter to cascading failures and to the interaction between overload, retries, and capacity [6]. The Amazon Builders' Library treats timeouts, retries, and backoff as a system-level concern and warns about retry amplification [7]. This literature is correct and widely read. What it does not provide is a model that says when composition is safe, which is what an operator needs in order to configure more than one mechanism at a time.

## III. Three Coupling Channels

We distinguish coupling channels by what the controllers share.

### A. Actuation coupling

Two controllers exhibit actuation coupling when their outputs move the same plant variable in the same direction.

A concurrency limiter and a rate limiter both bound offered load. A shedder and a rate limiter both bound admitted work. An autoscaler and an outlier detector both change effective serving capacity, in opposite directions. In each case the plant receives the sum of the corrections while each controller was tuned as though it were applying the whole correction alone.

The orchestrator specification studied in our companion work states this as an invariant it calls no double-tightening: an implementation that cuts a rate limit and reduces replica count for the same endpoint under mild error is judged to have two controllers fighting one disturbance. That an industrial specification names the failure is evidence that practitioners encounter it.

### B. Observation coupling

Two controllers exhibit observation coupling when one controller's actuation lies upstream of the other's measurement point, so that the second controller's input is a function of the first controller's output.

The autoscaler and shedder interaction of Section I is the canonical case. Shedding removes work before it reaches the point where request rate is measured, so the provisioning controller reads a demand signal that the protection controller is actively suppressing. The same structure appears whenever a protective mechanism is deployed at the edge of a measurement boundary, which is the normal place to deploy one.

Observation coupling is the most dangerous of the three because it is the least visible. Actuation coupling at least produces a plant that responds too strongly, which an operator may recognize as overshoot. Observation coupling

produces a controller that is behaving correctly on false data.

### C. Topological coupling

Two controllers exhibit topological coupling when one controller's action redistributes load and thereby changes the state the other controller observes at a different endpoint.

Ejection and failover routing are both redistributive. Removing an endpoint does not remove its traffic; it moves it. If survivors were already near saturation, the redistribution pushes them past the threshold that triggered the first ejection, and the process compounds. Retry amplification is a temporal variant: a timeout reduction converts slow successes into failures, failures into retries, and retries into additional offered load, so a tightening action increases the very quantity it was meant to reduce.

## IV. A Model of Overlapping Authority

We model the fleet around an operating point and treat each coupling channel in turn. The models are deliberately minimal. Their purpose is to identify the structural condition under which composition fails, not to predict trajectories.

### A. Setup

Let $x(t)$ denote a scalar congestion state at an endpoint, for instance queue depth or latency deviation from setpoint, normalized so that $x = 0$ is the operating point. Let controllers $i = 1, \dots, m$ observe $x$ and apply corrections $u_i$. Each controller has an actuation delay $\tau_i > 0$ comprising its measurement window, its decision interval, and the physical time for its action to take effect. For an autoscaler $\tau$ includes container start time and is often tens of seconds; for a shedder it is milliseconds.

### B. Actuation coupling breaks the delay margin

Consider $m$ controllers with overlapping authority, each applying proportional feedback $u_i = -g_i x(t-\tau)$ with common delay $\tau$ and gain $g_i > 0$. Acting alone, controller $i$ closes the loop

$$\dot{x}(t) = -g_i\, x(t-\tau) + d(t),$$

which is asymptotically stable if and only if $g_i\tau < \pi/2$, the classical delay margin for first-order feedback with pure delay. Define controller $i$'s margin fraction

$$\rho_i \;=\; \frac{g_i\tau}{\pi/2} \;\in\; (0,1).$$

A controller tuned to consume half its delay margin has $\rho_i = 1/2$.

Proposition 1 (Gain superposition). If the $m$ controllers act concurrently on the same plant variable, the composed loop is

$$\dot{x}(t) = -\Big(\textstyle\sum_{i=1}^{m} g_i\Big)\, x(t-\tau) + d(t),$$

which is unstable whenever $\sum_{i=1}^{m} \rho_i \geq 1$. In particular, any two controllers each tuned to consume half or more of the delay margin compose to an unstable loop, despite each being stable in isolation.

Proof sketch: Superposition of the corrections gives the stated composed loop, whose stability condition is $(\sum_i g_i)\,\tau < \pi/2$. Dividing by $\pi/2$ yields $\sum_i \rho_i < 1$. Negating gives the claim. ∎

Corollary 1 (Local tuning cannot prevent this). The margin fraction $\rho_i$ is determined by controller $i$'s own configuration and the plant. The stability condition depends on $\sum_j \rho_j$. No configuration surface enumerated in Section V exposes $\rho_j$ for $j \neq i$, nor any proxy for it. The condition is therefore not evaluable, let alone enforceable, at any individual controller.

Corollary 1 is the practical heart of the actuation-coupling argument. The failure is not that operators tune badly. It is that the quantity governing stability is not exposed anywhere an operator could read it. Adding a second well-tuned controller to a well-tuned system is an unsafe operation, and nothing in the tooling says so.

Unequal delays complicate the algebra without changing the conclusion. When $\tau_i$ differ, the composed characteristic equation $s + \sum_i g_i e^{-s\tau_i} = 0$ replaces the single-delay form, and the stability region shrinks relative to the single-controller case for the same reason. We use the common-delay case for exposition.

### C. Observation coupling induces positive feedback

Now consider two controllers with different roles. A fast controller $F$ reduces offered load with output $u_F \geq 0$, and a slow controller $S$ provisions capacity from a measured demand signal. Crucially, $F$ acts upstream of $S$'s measurement point, so $S$ observes

$$\hat{\lambda} \;=\; \lambda - u_F$$

rather than the true offered load $\lambda$. Let capacity be $\mu = \mu_0 + u_S$ with $u_S = g_S(\hat{\lambda} - \mu)$, let $u_F = g_F\, x$ with $g_F > 0$, and let congestion respond to capacity with sensitivity $\kappa = -\partial x/\partial \mu > 0$.

Proposition 2 (Signal masking). The composed system contains a feedback loop through the path $x \to u_F \to \hat{\lambda} \to u_S \to \mu \to x$ whose loop gain is $+\,g_F\, g_S\, \kappa$. The loop is positive, and the operating point is unstable when $g_F g_S \kappa > 1$.

Proof sketch: Trace the signs. An increase in congestion increases $u_F$ by $g_F > 0$. An increase in $u_F$ decreases $\hat{\lambda}$ by construction. A decrease in $\hat{\lambda}$ decreases $u_S$ by $g_S > 0$, hence decreases $\mu$. A decrease in $\mu$ increases $x$ by $\kappa > 0$. The product of the four sensitivities is positive, so the loop reinforces rather than opposes the initial perturbation, and the standard small-gain argument gives instability once the product exceeds unity. ∎

Corollary 2 (Detuning does not remove the loop). Reducing $g_F$ or $g_S$ reduces the loop gain but leaves the loop positive. Any configuration with both controllers active and $F$ upstream of $S$'s measurement retains a positive feedback path. Restoring negative feedback requires severing the dependence of $\hat{\lambda}$ on $u_F$, which is a structural change to the signal path rather than a tuning change.

Corollary 2 explains an experience many operators report: tuning the autoscaler's stabilization window makes the oscillation slower without making it go away. The window changes $g_S$, and $g_S$ is not the problem.

### D. Topological coupling has a branching threshold

Let an endpoint be ejected and its load redistributed over the surviving members of its pool. Let $\Gamma$ denote the expected number of additional endpoints driven past the ejection threshold by the redistribution from one ejection, given the current load margin distribution across the pool.

Proposition 3 (Cascade subcriticality). Treating ejections as a branching process with mean offspring $\Gamma$, the ejection cascade terminates almost surely if and only if $\Gamma < 1$. When $\Gamma > 1$ the cascade reaches the pool's live-endpoint floor with positive probability.

The practical content is that $\Gamma$ is computable before acting, from the topology and the current per-endpoint headroom, which is what makes topological coupling the most tractable of the three channels. The orchestrator specification's cascade-risk invariant is precisely a bound on a quantity of this form.

## V. Does the Premise Hold in Practice?

The model predicts trouble under two conditions: overlapping authority, and independent configuration with no shared signal. This section establishes that both conditions hold in software that is widely deployed. We use three kinds of public evidence and deliberately avoid proprietary incident data, so that every claim here is checkable by a reader.

### A. Configuration surface audit

Method: for each mechanism, read the upstream documentation and configuration reference, and record four fields. The signal the mechanism reads. The plant variable it actuates. The tuning parameters it exposes. Whether any documented hook allows it to observe or defer to a peer mechanism. Table I gives the schema and the mechanisms in scope. We audit Kubernetes horizontal pod autoscaling and cluster autoscaling, Envoy outlier detection, circuit breakers, adaptive concurrency, local and global rate limiting, and retry budgets, Istio destination-rule outlier detection, and Linkerd retry budgets.

We state the expected finding as a hypothesis rather than a result, since the audit is in progress: we expect the coordination-hook column to be uniformly empty, and we expect at least three pairs of mechanisms in the table to actuate the same plant variable. If either expectation fails the paper's premise weakens, and we will report that.

### B. Incident catalogue

Public postmortems establish that these interactions occur outside simulation. We use them for existence and mechanism, not for prevalence. This is a deliberate methodological limit. Vendors publish selectively, and most writeups lack the detail needed to diagnose a controller interaction, so any rate computed over published incidents would be an artifact of disclosure policy rather than a property of systems.

Method: rather than inherit a third-party curated list, whose maintenance we cannot depend on, we index primary sources directly. We search vendor incident archives from major cloud and infrastructure providers, apply stated inclusion criteria, and code each admitted incident against the three channels of Section III. Both authors code independently and we report agreement. The resulting index, with inclusion criteria and per incident coding, is released as an artifact so that the catalogue does not inherit our own maintenance either.

We also draw on research corpora that have publications behind them and are therefore more durable than community lists, including published studies of cloud fault life cycles and of user- and provider-reported cloud failures.

### C. Practitioner literature

Independent of any dataset, the interactions we model are already named in the canonical operational literature. The Google SRE treatment of cascading failures describes overload propagation and the role of retries in amplifying it [6]. The Amazon Builders' Library treats retry amplification and load shedding as system-level concerns rather than local ones [7], [8]. We cite this literature as evidence that the phenomena are recognized, and position our contribution as supplying the model and the remedy that it does not.

## VI. Concerto: Arbitration by Construction

Concerto is an arbitration layer that sits between the controllers and the plant. Controllers become advisors: they compute desired corrections, and Concerto decides what is actually applied. The name reflects the intent, which is controllers playing from one score rather than each at full volume.

The design has one principle per coupling channel, plus a cross-cutting scheduling rule.

### A. P1: bounded shared authority

Addresses actuation coupling.

Concerto computes a single severity estimate $s \in [0, 1]$ from the plant state, once, and derives a total corrective authority budget $B(s)$. Each actuator receives an allocation $\alpha_i(s)$ subject to $\sum_i \alpha_i(s) \leq B(s)$. Allocations are assigned by actuation cost and time constant, not by letting each controller take what it wants.

TABLE I
Configuration audit schema. Mechanisms in scope; the coordination-hook column is the finding the model predicts will be uniformly empty.

| Mechanism | Reads | Actuates | Coord. hook |
|---|---|---|---|
| K8s HPA | utilization | replicas | — |
| K8s cluster autoscaler | pending pods | nodes | — |
| Envoy outlier detection | errors, latency | serving set | — |
| Envoy circuit breakers | in-flight, pending | admitted load | — |
| Envoy adaptive concurrency | latency | concurrency | — |
| Envoy rate limiting | request rate | admitted load | — |
| Envoy retry budget | failure rate | offered load | — |
| Istio outlier detection | errors | serving set | — |
| Linkerd retry budget | failure rate | offered load | — |

The structural property is that total loop gain is bounded by $B$ regardless of how many controllers are enabled. Enabling an additional controller redistributes authority rather than adding to it, so the margin-fraction sum of Proposition 1 is bounded by construction and Corollary 1 no longer bites: the governing quantity is computed in one place, by the component that owns it.

This is control allocation in the aerospace sense [1], applied to a plant whose actuators happen to be named autoscaler and shedder.

### B. P2: counterfactual signal reconstruction

Addresses observation coupling.

Each controller is presented with the signal it would observe if no other controller were acting. Concretely, the provisioning controller reads $\lambda = \hat{\lambda} + u_F$, the sum of admitted and shed load, rather than admitted load alone. Generalizing, Concerto maintains for each exposed signal an accounting of the corrections applied upstream of its measurement point, and serves the reconstructed pre-intervention value.

By Corollary 2 this is the only class of remedy that works, because it removes the dependence of $\hat{\lambda}$ on $u_F$ rather than shrinking it. The implementation cost is low: sheddders and rate limiters already count rejected work for their own telemetry, so the required quantity usually exists and is merely not wired to the consumer that needs it.

### C. P3: topology-gated action admission

Addresses topological coupling.

Before applying any redistributive action, Concerto computes the projected cascade gain $\Gamma$ of Proposition 3 from the current topology and per-endpoint headroom, and admits the action only if the post-action $\Gamma$ remains below a subcriticality threshold. When the full requested action set would exceed it, the set is scaled back rather than dropped, preserving as much protective value as the threshold allows.

### D. P4: time-scale separation

Cross-cutting.

Even under a shared budget, two actuators can chase the same disturbance component. Concerto splits the severity signal by frequency band and assigns bands to actuators by time constant: the fast band drives shedding and concurrency, the slow band drives replica provisioning. An actuator never receives authority over a disturbance component faster than its own actuation delay, which prevents a slow controller from reacting to transients it cannot affect in time, a known source of autoscaler thrash.

## VII. Implementation

We implement Concerto in two settings.

The first is a sidecar and controller pair for Kubernetes and Envoy. The severity estimator and allocator run as a cluster-scoped controller. Existing mechanisms are placed in advisory mode where their configuration permits it, and where it does not, they are bounded externally by the allocator setting their limits each interval. Signal reconstruction is implemented by exposing a reconstructed demand metric that the autoscaler consumes in place of the raw one, which requires no change to the autoscaler itself.

The second is a reference implementation inside a fleet simulator, used for controlled ablation at scale where testbed experiments are impractical. The simulator exposes the same public decision surface used in our companion work on grading long-horizon controllers, which lets the same arbitration logic be evaluated under both.

A deliberate design constraint is that Concerto requires no modification to the controllers it arbitrates. Each of the three principles is implementable by bounding a controller's output, substituting its input, or gating its action, all of which are available from outside.

## VIII. Evaluation Protocol

This section states the campaign we are running. It reports no results.

### A. Testbed

An open microservice benchmark on Kubernetes with Envoy or Istio providing outlier detection, retries, and concurrency limiting, and horizontal pod autoscaling providing capacity. We favor DeathStarBench as the primary

application for its established use in the systems community [9], with a lighter alternative retained for iteration speed. Workloads are driven by an open-loop generator, with arrival patterns taken from public cluster traces to obtain realistic non-stationarity.

The testbed carries the paper's empirical weight. The central experiment is the observation-coupling demonstration, because it is the channel that retuning cannot address: we aim to produce the autoscaler and shedder oscillation of Section I on unmodified, reasonably configured, widely deployed software.

### B. Simulator

A fleet simulator supplies controlled ablation across many scenarios and at scales the testbed cannot reach. It is the instrument for isolating mechanism, not for establishing that the phenomenon is real; the testbed does that.

### C. Baselines

B1, independent defaults. Every mechanism enabled with vendor default configuration. This is not a strawman. It is what a team gets by following each mechanism's own documentation.

B2, independently retuned. Each controller detuned by an expert to compensate for observed instability, without any shared signal. This baseline tests whether the problem is a tuning problem, and Corollary 2 predicts it will fail specifically on observation coupling while improving actuation coupling.

B3, strict precedence arbiter. A naive arbitration scheme in which a fixed priority order determines which controller acts when several wish to. This isolates the value of proportional allocation over simple mutual exclusion.

B4, Concerto.

### D. Metrics and research questions

Metrics: oscillation amplitude and period after a step disturbance, settling time, objective attainment, capacity cost, cascade incidence, goodput under a retry storm, and the rate of simultaneous tightening actions on one endpoint.

RQ1. Does each coupling channel reproduce on unmodified deployed software under reasonable configuration?

RQ2. Does expert retuning remove instability from actuation coupling but not from observation coupling, as Corollary 2 predicts? This is the sharpest falsifiable prediction the model makes and we treat it as the paper's central test.

RQ3. Does Concerto restore stability without a cost or attainment penalty relative to the best-performing baseline?

RQ4. What does each principle contribute? Measured by removing P1 through P4 individually.

RQ5. How does arbitration behave as the number of enabled mechanisms grows? Proposition 1 predicts the independent baselines degrade with $m$ while Concerto does not.

TABLE II
Results table structure. No values are reported; the campaign is in progress.

| Configuration | Osc. | Settle | Attain. | Cost |
|---|---|---|---|---|
| B1 defaults | — | — | — | — |
| B2 retuned | — | — | — | — |
| B3 precedence | — | — | — | — |
| B4 Concerto | — | — | — | — |
| less P1 | — | — | — | — |
| less P2 | — | — | — | — |
| less P3 | — | — | — | — |
| less P4 | — | — | — | — |

## IX. Related Work

Control allocation. The assignment of corrective effort among redundant actuators is well developed in aerospace control [1], [2]. Our contribution is not to that theory but to the observation that service fleets have the same actuator redundancy and none of the allocation discipline, and to the two coupling channels, observational and topological, that the aerospace formulation does not address because aircraft control surfaces do not corrupt each other's sensors or redistribute load across a graph.

Autoscaling. Autoscaling has been studied extensively as a control and prediction problem [3]. This work assumes the autoscaler is the controller closing the loop. We study what happens when it is not the only one, and Proposition 2 concerns the validity of the demand signal such work takes as given.

Overload control and admission. Overload protection for servers has a long lineage [4], and adaptive concurrency limiting imports congestion-control ideas into the application layer [5]. Again the composition is out of scope in that literature.

Cascading failure. Cascade dynamics have been modeled in power grids and in networks, and the branching-process threshold of Proposition 3 is standard there. The contribution here is to treat cascade risk as an admission gate on a controller's action rather than as a post-hoc analysis.

Practitioner literature. The operational canon [6], [7], [8] names these interactions and prescribes remedies per mechanism. We supply the composition model and an arbitration layer, which it does not.

## X. Threats to Validity

Model fidelity. The models of Section IV are linearized and scalar. They are intended to identify structural conditions, not to predict trajectories, and the testbed is the check on whether the conditions they identify actually govern real systems.

Configuration realism. A reviewer may object that a competent operator would not deploy the configurations in

baseline B1. The audit of Section V is partly a response: we report what the defaults are and what the documentation recommends, so that the baseline is defensible by citation rather than by assertion.

Benchmark representativeness. Findings on a benchmark application may not transfer to production topologies, which are larger and more irregular. The simulator partially addresses scale and the audit addresses mechanism coverage, but neither substitutes for production validation, which we do not have.

Incident catalogue bias. Addressed directly in Section V. We make no prevalence claim, and the catalogue supports existence and mechanism only.

## XI. Conclusion

Service fleets run many reliability controllers over one plant and allocate authority among them not at all. We identified three channels through which those controllers interfere, and showed that each has a distinct structural signature: summed gains that break a delay margin no controller can observe, a masked signal that turns negative feedback positive and cannot be retuned away, and a redistribution process with a branching threshold. We presented an arbitration layer with one principle per channel, implementable without modifying the controllers it governs.

The broader claim is that reliability mechanisms should be designed for composition and are not. Every mechanism in the audit of Section V is individually sound and individually well documented, and not one of them exposes the information a neighboring controller would need in order to be safe. That is a gap in how the mechanisms are specified, not in how operators use them.

## References


[1] W. C. Durham, "Constrained control allocation," Journal of Guidance, Control, and Dynamics, vol. 16, no. 4, pp. 717–725, 1993.

[2] T. A. Johansen and T. I. Fossen, "Control allocation: A survey," Automatica, vol. 49, no. 5, pp. 1087–1103, 2013.

[3] T. Lorido-Botrán, J. Miguel-Alonso, and J. A. Lozano, "A review of auto-scaling techniques for elastic applications in cloud environments," Journal of Grid Computing, vol. 12, no. 4, pp. 559–592, 2014.

[4] M. Welsh and D. Culler, "Adaptive overload control for busy internet servers," in USENIX Symposium on Internet Technologies and Systems (USITS), 2003.

[5] Netflix Technology Blog, "Performance under load: Adaptive concurrency limits," 2018. [Online]. Available: https://netflixtechblog.medium.com/

[6] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media, 2016, ch. 22, "Addressing Cascading Failures."

[7] M. Brooker, "Timeouts, retries, and backoff with jitter," Amazon Builders' Library. [Online]. Available: https://aws.amazon.com/builders-library/

[8] D. Yanacek, "Using load shedding to avoid overload," Amazon Builders' Library. [Online]. Available: https://aws.amazon.com/builders-library/

[9] Y. Gan et al., "An open-source benchmark suite for microservices and their hardware-software implications for cloud and edge systems," in ASPLOS, 2019.